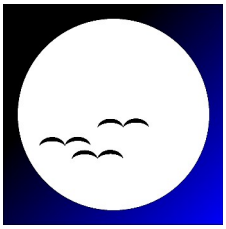

The QMdbSim2 Project

Device Developer Documentation

Document History

2026.07.07 – Initial documentation. Caveat emptor.

Prerequisites

If you haven't already done so, please start with the QMdbSim2 Overview and Basic User PDFs.

I assume that you have experience using Microchip MPLabX or similar tools to create micro-controller device code.

Parts of this project use Java. Pre-compiled JAR files are provided so you don't need to know anything about Java unless you wish to modify/compile the simulator interface code. If so, see the API Developer documentation.

Overview

To run a QSpice simulation of a specific Microchip micro-controller device, we need certain device-specific files. Device Developers create these files. The process is super-simple using a dedicated command-line tool, `QSymGen3`.

Here's the short version:

- Download a copy of the relevant device datasheet.
- Create a text file that includes the pin names and types taken from the datasheet (i.e., a "Pin Definitions File").
- Run `QSymGen3` against the text file to produce the required device-specific symbol, schematic, and DLL source files.
- Compile the DLL file along with the `QMdbSim2` shared DLL code sources.

That's all that's technically required. You should, of course, do some testing so you'll need to create an example top-level schematic and a Device Program. Maybe create some documentation. And, hopefully, send me a copy to share with the community.

Required Development Tools

Assuming that you have already set your computer up as described for Basic Users, you'll also need:

- The `QSymGen3` tool. I have included a copy of the compiled program in the `QSymGen3` folder for this project. The full tool sources are in the QParser2 Project section of the repository if you'd like to compile them yourself.
- The `QMdbSim2_Shared` and `Developer_Demo` folder files included with this project.
- A 32-bit Java JDK, v8 or newer to compile Device DLLs using Java Native Interface (JNI). (See the Basic User PDF for instructions to get the JDK.)
- A modern C++20 compiler to compile the DLL(s). I used the free community version Microsoft Visual Studio 2026 IDE/toolset. (The DMC compiler shipped with QSpice will not work.)

With those requirements in hand, grab a copy of the `Developer_Demo` folder.

The Pin Definitions File

A Pin Definitions file is used to create the device-specific symbol, schematic, and DLL code using the `QSymGen3` tool (part of the QParser2 Project).

The demonstration files are what I used to create `QMdbSim2` files for ATtiny25/45/85 devices. It took maybe a half-hour to create the Pin Definition file from the datasheet. Open `ATtinyx5.qpins` in a text editor to follow along.

The format of the file should be clear. Each non-comment line begins with a single-character (record type) followed by additional text (if required). Some record types are "Control Types" and some are "Pin Definitions."

Control Records

Record Type: D (Description)

Required: No

Example: `D ATtiny25/45/85 8-Bit AVR Microcontroller`

Purpose: Text displayed in the Description field of the Symbol Properties pane. Recommended use: List of supported devices or description of device family.

Record Type: P (Part Numbers)

Required: Yes

Example: `P ATtiny25 ATtiny45 ATtiny85`

Purpose: Part numbers supported by this Device Symbol/Schematic/DLL. Must be separated by spaces. These part numbers will be included in a drop-down list in the Symbol Properties pane for the Device Symbol.

Notes: ***Device part numbers are case-sensitive!*** For example, ATTINY85 and ATtiny85 are not the same and only the latter is recognized by the Microchip software simulator. Because QSpice down-cases this text in the netlist, there is a lookup in the Device DLL that maps the

passed text to the values as specified here. (See Simulator Peripheral Support By Device for the official list.)

Record Type: R (Output Pin/Port Internal Resistance)

Required: No

Example: `R 1`

Purpose: Sets the underlying DLL port `ROUT` value. (See QSpice documentation.)

Notes: If omitted, the default value is 1R. Output ports are voltage sources and probably should not be used to limit currents/produce voltage drops in `QMdbSim2` devices. See discussion about electrical fidelity in Basic User documentation.

Pin Definitions

Record Type: V (Supply Voltage Reference)

Required: Yes

Example: `V VCC`

Purpose: Sets the name of the supply voltage pin as defined in the datasheet.

Notes: Different Microchip devices appear to use different names for the supply pin. Use the name from the datasheet.

The voltage on this pin is used during simulation initialization to set the device supply voltage. After initialization, changes to the voltage on this pin are ignored since the underlying Microchip software simulator does not appear to support dynamic supply voltage changes.

Record Type: G (Ground Voltage Reference)

Required: Yes

Example: `G GND`

Purpose: Sets the name of the ground voltage pin as defined in the datasheet.

Notes: Different Microchip devices appear to use different names for the ground pin. Use the name from the datasheet. If the pin name is `GND` (as it is here), the generated code pin name will be "`GND_`" to prevent QSpice confusion.

Internally, this is used as the "DLL Gnd" port type.

Record Type: I (Input Pin)

Required: No

Example: `[None in this example]`

Purpose: Generates an Input-only pin. The name of the pin must match the datasheet. Subsequent text is optional; it is included in small text below the pin name in the Device Symbol. Any

optional text is merely decorative (no functional implications) but is useful to remind user of alternate pin functions.

Notes: Input pins link a symbol pin directly to a DLL input port.

Record Type: O (Output Pin)

Required: No

Example: `[None in this example]`

Purpose: Generates an Output-only pin. The name of the pin must match the datasheet. Subsequent text is optional; it is included in small text below the pin name in the Device Symbol. Any optional text is merely decorative (no functional implications) but is useful to remind user of alternate pin functions.

Notes: Output pins link a symbol pin directly to a DLL output port.

Record Type: B (Bi-directional/GPIO Pin)

Required: No

Example: `B PB0 MOSI/DI/SDA/AIN0/OC0A/OC1A/AREF/PCINT0`

Purpose: Generates a bi-directional symbol pin. The name of the pin must match the datasheet. Subsequent text is optional; it is included in small text below the pin name in the Device Symbol. Any optional text is merely decorative (no functional implications) but is useful to remind user of alternate pin functions.

Record Type: K (Simulation Clock Pin)

Required: Yes

Example: `K SIMCLK`

Purpose: Generates a symbol pin designated to drive the software simulation clock by a single instruction cycle. This is not a physical device pin so pin name is arbitrary (SIMCLK is recommended). Any text following the pin name is merely informational and displayed in small text below the pin name.

Record Type: X (Skip Pin / Spacer)

Required: No

Example: `X`

Purpose: Consumes space in the symbol but produces no pin. Used for spacing pins in the symbol.

Notes: The order of pin records determines the order of pins in the symbol. Two columns are created with the first half of the pin declarations on the left side of the symbol and the second half on the right side. Insert X records judiciously to add spacing between pins and push pins from the left column to the right column.

Generating Device Files

The `QSymGen3` command-line tool (part of the QParser2 project) processes the Pin Definitions file. It also requires a Device DLL template file, `QMdbSim2_Dll_Template.cpp`, which may be in the working directory or in `QSymGen3.exe` folder. These two files are in the `QSymGen3` subfolder of the example folder for convenience. (Note: Do not edit the DLL template file unless you have API Developer ambitions.)

Open a command prompt in the `Development_Demo` folder and run the following command:

```
.\QSymGen3\QSymGen3.exe ATtinyx5.qpindef
```

This will generate the following files (prompting before overwriting):

```
ATtinyx5.qsym       ← The Device Symbol file
ATtinyx5.qsch       ← The Device Schematic file
ATtinyx5.cpp        ← The Device DLL source file
```

Note that the base name of the output file is taken from the Pin Definitions file name. Case is preserved.

The generated files do not require further editing; all device-specific text and values are applied to the files. If a newer version of the DLL template or `QSymGen3` becomes available – or if you need to change the Pin Definitions file – simply re-generate the files with `QSymGen3`.

The final step is compiling the Device DLL. (And, of course, testing. And sharing. Don't forget to share.)

Compiling The Device DLL

The device DLL code must be compiled with the `QMdbSim2_Shared` sources and the Java JDK. Getting the compile environment set up the first time is, perhaps, a bit tricky. But once it's set up, compiling future device projects is easy.

The project repository files include MSVS 2026 solution/project files that you can use. If you use a different tool-chain, here are the key configurations:

- Set up to compile for 32-bit Windows DLL.
- Add the `QMdbSim2_Shared` code folder to the project. You need to compile the `*.cpp` files into the project and the compiler must also be able to locate the shared header files.
- Add the Java JDK folders to "Additional Include Directories." For example:

  ```
  C:\Program Files\Eclipse Adoptium\jdk-8.0.492.9-hotspot\include
  C:\Program Files\Eclipse Adoptium\jdk-8.0.492.9-hotspot\include\win32
  ```

  The above assumes that you have installed the 32-bit Eclipse Adoptium Java 8 JDK to the default location.

Hopefully, that's it. Let me know if I missed something.

Testing Device Files

To test the device files, you'll need to do the Basic User stuff including:

- Drag the Device Symbol onto a new QSpice top-level schematic.
- Wire up the schematic to appropriate test circuitry including DC voltage supply/ground and a `SIMCLK` pulse voltage source.
- Set the `SIMCLK` source to match the Device Program instruction frequency.
- Create a Microchip device program appropriate to the circuit and compile it (`*.hex`/`*.elf`).
- Configure the Device Symbol properties in the Symbol Properties pane.

And test.

Note: Most of the MSVS project files are set up for debugging into the DLL. Configuration changes may be required….

Sharing Device Files

Once you have developed new Device Files, I hope that you'll share them. If you send me your device files, I'll add them to the `QMdbSim2` repository (with, of course, credit to you).

Thanks in advance for supporting the community!

Additional Tools

I have included a couple of potentially useful tools in the project's Devices folder:

MSVS_Device_Templates

If you are using MSVS 2026, you can copy the `QMdbSim2_Device_Template.zip` to your user Templates folder. (See MSVS documentation.) Once there, you can add a new project that is pre-configured for compiling Win32 DLLs using the `QMdbSim2_Shared` folder, etc. If you keep the project's structure and MSVS solution/project files, it might work for you. Or not. Let me know.

Claude_Prompts

I tried using Claude to generate Pin Definitions files directly from datasheets. It seemed to work better than I expected. The folder contains the "playbook" and some instructions. Of course, you'll need to carefully review the results. Let me know if you try it and, if you develop more complete Claude prompts, please share them.

Resources

My GitHub QSpice Repository

https://github.com/robdunn4/QSpice

In addition to this project, you'll find information about C-Block component concepts ("C-Block Basics"), various C-Block component examples, tips on using VSCode and Microsoft Visual Studio (MSVS) IDEs for C-Block development and debugging, and links to other useful QSpice-related sites/repositories.

Microchip

MPLabX IDE (free)

Microchip MPLabX SDK / MDBCore API

Microchip Debugger (MDB) User's Guide (one)

Microchip Debugger (MDB)User's Guide (another one)

Simulator Peripheral Support By Device

Microsoft

Visual Studio Downloads (2026 Community Edition is free)

Java JDK/JRE

Eclipse Adoptium Temurin (free open source Java JDK/JRE)

Conclusion

I hope that you find this project to be useful or – at least – interesting. Please let me know if you find problems or suggestions for improving the code or this documentation.

You can find me on the QSpice Forums as @RDunn.

--robert